

Here is the complete, standalone blueprint for your **SIM Module Emergency Call & SMS Setup**. This guide covers the physical wiring (including the safe TP4056 lithium battery isolation system), the Python code updates, and the Home Assistant configuration.

You can save this entire response directly into a PDF for your future installation.



Hardware Architecture & Safety Summary

- **GSM Module:** Standard SIM800L (or compatible cellular breakout).
- **Power Engine:** 1× Brand-New high-quality 18650 Li-ion battery (degraded/old batteries will sag and fail during transmission spikes).
- **Charging IC:** TP4056 Lithium Charging Module.
- **Safety Isolation Strategy:** The Raspberry Pi 5 runs a steady 5V current into the TP4056 input. The SIM module is wired directly to the battery/charger outputs *in parallel*. This fully shields the Pi from the module's massive **2 Amp transmission bursts** while maintaining a safe, unified ground loop.



CRITICAL ELECTRICAL WARNING: Never bridge the 3.7V battery's positive terminal directly to any 5V or 3.3V GPIO pin on the Raspberry Pi. Doing so will overcharge the battery (causing fire/explosion risks) or instantly kill the Pi's internal voltage regulators.



Phase 1: Physical Wiring Diagram & Pinouts

Ensure all connections are soldered or pinned securely according to this roadmap:

1. The Power & Charging Loop

- **Pi 5V (Physical Pin 2 or 4)** $\rightarrow$ Connect to **IN+** on the TP4056 board.
- **Pi GND (Physical Pin 6)** $\rightarrow$ Connect to **IN-** on the TP4056 board.
- **Battery Positive (+)** $\rightarrow$ Connect to **B+** (or BAT+) on the TP4056.
- **Battery Negative (-)** $\rightarrow$ Connect to **B-** (or BAT-) on the TP4056.
- **SIM Module VCC** $\rightarrow$ Connect directly to the **B+** pad on the TP4056 (parallel with the battery).
- **SIM Module GND** $\rightarrow$ Connect directly to the **B-** pad on the TP4056 (parallel with the battery).

2. The Serial Data Loop

- **SIM Module TXD** $\rightarrow$ Connect to **Pi RXD** (GPIO 15 / Physical Pin 10).
- **SIM Module RXD** $\rightarrow$ Connect to **Pi TXD** (GPIO 14 / Physical Pin 8).
- *Note: Because the Pi's ground meets the charger at IN-, and IN- shares a internal trace with B-, the Pi and SIM module automatically establish the required common ground loop.*



Phase 2: Raspberry Pi OS Serial Port Preparation

By default, Debian uses the hardware serial port for an operating system login terminal. We must disable this terminal so your custom Python script can control the port.

1. SSH into your Raspberry Pi and run:

```
sudo raspi-config
```

2. Navigate to **Interface Options** $\rightarrow$ **Serial Port**.
3. When asked: *Would you like a login shell to be accessible over serial?* $\rightarrow$ Select **No**.
4. When asked: *Would you like the serial port hardware to be enabled?* $\rightarrow$ Select **Yes**.
5. Save, exit, and reboot the system:

```
sudo reboot
```

Phase 3: Complete Python Automation Script

This script includes your original telemetry loop, the automatic sensor watchdog triggers, and the **new local HTTP Webhook server** that processes manual SOS commands sent from Home Assistant.

Create or update your script file (e.g., nano ~/security_system.py):

```
# -*- coding: utf-8 -*-
import time
import math
import datetime
import subprocess
import threading
import serial
import requests
import smtplib
from http.server import BaseHTTPRequestHandler, HTTPServer
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

# =====
# SYSTEM & EMERGENCY CONFIGURATION
# =====
HA_URL = "http://192.168.0.40:8123"
HA_TOKEN = "YOUR_HA_LONG_LIVED_ACCESS_TOKEN"
```

```
TELEGRAM_BOT_TOKEN = "YOUR_TELEGRAM_BOT_TOKEN"
```

```
TELEGRAM_CHAT_ID = "YOUR_TELEGRAM_CHAT_ID"
```

```
EMERGENCY_NUMBERS = ["+8801794684164", "+8801342570575"]
```

```
GMAIL_USER = "nabilredwoan2005@gmail.com"
```

```
GMAIL_APP_PASSWORD = "uexpklsjjfuqxhkt" # Clean app password space
```

```
ALERT_RECIPIENTS = ["redwannabil116@gmail.com"]
```

```
# =====
```

```
# HARDWARE PORT INITIALIZATION
```

```
# =====
```

```
# Uses the hardware mapped port /dev/serial0
```

```
sim = serial.Serial('/dev/serial0', 9600, timeout=1)
```

```
alarm_active = False
```

```
# =====
```

```
# SIM MODULE CRITICAL EMERGENCY CORE
```

```
# =====
```

```
def emergency_sos
```

```
    """Executes consecutive SMS alerts and live call sequences over AT commands"""
```

```
    for number in EMERGENCY_NUMBERS:
```

```
        try:
```

```
            print(f"[SOS] Targeting: {number}")
```

```
            # 1. Initialize text message frame
```

```
            sim.write(b'AT+CMGF=1\r\n')
```

```
            time.sleep(0.5)
```

```
            # 2. Stage destination directory routing
```

```
            sim.write(f'AT+CMGS="{number}"\r\n'.encode())
```

```
            time.sleep(0.5)
```

```
            # 3. Write payload text and append Hex 1A (Ctrl+Z) to send
```

```
            sim.write(message.encode() + b'\x1a')
```

```
            time.sleep(4)
```

```
            print(f"[SOS] SMS dispatched to {number}. Initializing voice call link...")
```

```
            # 4. Dial emergency route line
```

```
            sim.write(f"ATD{number};\r\n".encode())
```

```
            time.sleep(20) # Keep line open for 20 seconds to force loud ring vibration
```

```
            # 5. Terminate voice channel link
```

```
            sim.write(b"ATH\r\n")
```

```

        time.sleep(1)
    except Exception as e:
        print(f"[SIM ERROR] Failed executing emergency pipeline for {number}: {e}")

def reset_alarm_lock
    global alarm_active
    time.sleep(60) # Protects system from duplicate trigger looping within 1 minute
    alarm_active = False

def trigger_full_alarm
    global alarm_active
    if not alarm_active:
        alarm_active = True
        print(f"[ALARM ACTIVATE] Reason: {message}")
        threading.Thread(target=reset_alarm_lock, daemon=True).start()
        threading.Thread(target=emergency_sos, args=(message,), daemon=True).start()
        # Add background threads here for Telegram, Gmail, and Buzzers as needed

# =====
# HOME ASSISTANT MANUAL SOS WEBHOOK INTERFACE
# =====
class SOSWebHookHandler(BaseHTTPRequestHandler):
    def do_POST
        if self.path == '/trigger_manual_sos':
            msg = "🚨 MANUAL EMERGENCY SOS: Your home person is in danger! Please take
immediate action!"
            trigger_full_alarm(msg)

            self.send_response(200)
            self.send_header('Content-type', 'application/json')
            self.end_headers()
            self.wfile.write(b'{"status": "activated"}')
        else:
            self.send_response(404)
            self.end_headers()

def log_message    format
    return # Suppresses console output floods

def run_webhook_server
    server = HTTPServer(('0.0.0.0', 8089), SOSWebHookHandler)
    print(f"[SERVER] SOS Webhook Core active on port 8089...")
    server.serve_forever()

```

```
# Launch interface thread
threading.Thread(target=run_webhook_server, daemon=True).start()

# =====
# MAIN EXECUTION HOLD
# =====
if __name__ == "__main__":
    while True:
        time.sleep(1)
```

Phase 4: Home Assistant Integration

Now, link the dashboard interface to the script's incoming server port.

Step 1: Establish the Shell Target Command

1. Open your Home Assistant system folder setup:

```
nano ~/homeassistant/config/configuration.yaml
```

2. Find or create your top-level shell_command: tree entry and register the network trigger:

```
shell_command:
  trigger_manual_sos: "curl -X POST http://127.0.0.1:8089/trigger_manual_sos"
```

3. Save, close, and restart your Home Assistant container instance.

Step 2: Render the Safe Dash Card

Open your Home Assistant UI dashboard environment, choose **Add Manual Card**, and use this specific structure:

```
type: button
name: 🚨 TRIGGER EMERGENCY SOS 🚨
icon: mdi:alert-octagon
icon_height: 80
show_name: true
show_icon: true
show_state: false
```

```
tap_action:  
  action: none  
hold_action:  
  action: call-service  
  service: shell_command.trigger_manual_sos
```

Note: Binding this to hold_action ensures you must actively press and hold the button card layout for a full 2 seconds to fire the alarm, preventing accidental taps while scrolling your phone UI.